



Dynamic difficulty adjustment using a large language model: A case study in Magic: The Gathering

Xiaoxu Li ^a, Zifan Ye ^a, Yi Xia ^a, Ruck Thawonmas ^{b,*}

^a Graduate School of Information Science and Engineering, Ritsumeikan University, 2-150 Iwakura-cho, Ibaraki, 567-8570, Osaka, Japan

^b College of Information Science and Engineering, Ritsumeikan University, 2-150 Iwakura-cho, Ibaraki, 567-8570, Osaka, Japan

ARTICLE INFO

Keywords:

Large language model
Dynamic difficulty adjustment
Magic: The Gathering (MTG)
Difficulty arc

ABSTRACT

This paper presents a framework called LLM-MTG-DDA, which uses a large language model (LLM) in the real-world card game Magic: The Gathering (MTG) to act as a player and implement a dynamic difficulty adjustment (DDA) mechanism. LLMs, as a highly useful technology, have been explored across various fields. However, research on using LLMs for DDA in games, particularly in complex turn-based games, is very limited. In this paper, GPT-4o acts as two players in a simplified version of MTG. One GPT-4o player plays as a regular player (LLM-Player), while the other GPT-4o player adjusts its strategy based on the current game state to balance the difficulty (LLM-DDA). Our LLM-MTG-DDA framework, with a suitable objectives for different players, demonstrates reasonable DDA, with the LLM-Player's win rate and the win rate per round (excluding draws) both approaching 50%. This framework provides insights for applying LLMs as DDA mechanisms in other similar games.

1. Introduction

Exploring the ability of large language models (LLMs) to complete tasks across various domains has been widely recognized as a meaningful and valuable area of research. This is due to their exceptional language processing capabilities [1] and emergent abilities [2]. Given these strengths, LLMs are expected to tackle many challenges with appropriate prompts [1,3,4] that were difficult to overcome before their emergence. However, it is also important to acknowledge that in the process of using and exploring these models, verifying their feasibility and identifying their weaknesses in specific tasks are essential steps for their optimization and broader application. Therefore, we believe that neglecting the exploration of LLMs across different domains would be shortsighted.

Because of this, we can find many studies and explorations using LLMs across various fields, even though their performance is sometimes not as good as existing solutions. The same applies to the gaming domain. Many studies have explored how to utilize or optimize LLMs to accomplish various in-game tasks. For example, some research focuses on using LLMs to assist game designers in creating in-game layouts [5, 6], while others explore their role as agents [7,8] within the game or as interactive mechanisms [9–11] that engage with players. These studies suggest that LLMs could have extensive applications in gaming.

In a recent scoping review [12] on the use of LLMs in gaming, it was found that research on using them as in-game agents and AI accounts for the largest proportion. Moreover, recent Vision-Language Models (VLMs) have also demonstrated promising capabilities in game-related tasks [13,14]. For instance, VLMs show the ability to interact with complex action role-playing games using visual input [15]. This indicates that LLMs have significant potential as agents in various types of games.

In a previous study [16] on using LLMs to control a turn-based game where players compare dice values, a dynamic difficulty adjustment (DDA) mechanism—where one player adjusts their abilities based on the opponent's performance to maintain a balanced challenge—was successfully validated as highly feasible. However, this research did not demonstrate that LLMs can serve as DDA in a real-world game. Building on this research, we aim to extend its findings and recommendations to a real-world turn-based card game that differs from that previous study. We have chosen Magic: The Gathering (MTG), a popular card game. Our goal is to expand and optimize the application of LLMs in real card game scenarios by developing a framework where an LLM serves as the DDA mechanism in a turn-based card game. To evaluate our framework, we will conduct a detailed analysis of the log data from each game round and examine the overall difficulty curve of the game.

* Corresponding author.

E-mail addresses: xiaoxu.li@ice.ci.ritsumei.ac.jp (X. Li), gr0734sh@ed.ritsumei.ac.jp (Z. Ye), gr0666ih@ed.ritsumei.ac.jp (Y. Xia), ruck@is.ritsumei.ac.jp (R. Thawonmas).

<https://doi.org/10.1016/j.entcom.2025.100997>

Received 2 February 2025; Received in revised form 2 July 2025; Accepted 15 July 2025

Available online 25 July 2025

1875-9521/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

This paper makes three main contributions. First, we propose a framework that enables the use of LLMs as a DDA mechanism in MTG called LLM-MTG-DDA. Second, we identify a prompt necessary for utilizing LLMs as a DDA mechanism in MTG. Third, we introduce the concept of the difficulty arc as a method for evaluating the effectiveness of DDA. In addition, we explore what features define a good difficulty arc for DDA.

2. Related work

2.1. LLMs for DDA

There is currently very little research on using LLMs as a DDA mechanism in games. One previous study explored the use of an LLM to play Tic-Tac-Toe, finding that GPT-4 performed the best [17]. In parallel, other recent studies have explored the use of LLMs and VLMs as agents or assistants in strategic and card-based game environments. For example, CALYPSO [18] leveraged LLMs to assist Dungeon Masters in tabletop RPGs by maintaining game state and providing narrative suggestions, while VLMs were applied as visual-information-based coaches in robotic soccer scenarios [19]. In more structured settings, LLMs have been evaluated in decision-making tasks across games such as UNO [20], GuanDan [21], and Poker [22], often incorporating fine-tuning or reinforcement learning to improve performance. Notably, another recent work achieved near human-level play in Pokémon battles, where the PokéLLMon system reached a 49% win rate against human players while demonstrating strategically rich behavior [23], highlighting the feasibility of LLMs as competent agents in complex, real-world card-based games.

However, these studies did not address game difficulty. In another study [16], researchers created a turn-based game featuring a Dungeons & Dragons combat scenario to test whether GPT-4o could function as a DDA. They also identified highly promising prompts that enabled GPT-4o to serve effectively as a DDA. Building on these findings and methodologies, we extend this research by applying it to a real-world turn-based card game, MTG.

In one aforementioned study [16], the game scenario was a combat setting featuring two players. The outcome of each round was determined by rolling a six-sided die for each player, with the higher roll winning the round. The losing player received damage equal to the difference between the two dice rolls. In addition, each player started with 10 initial health points. The game ended when one player's health dropped to zero, and the player with remaining health was declared the winner.

We consider this design to be a very simple turn-based dice game, which is sufficient for verifying whether an LLM has the potential to serve as a DDA. However, applying an LLM to a more complex turn-based game requires deeper research and design. Given the proven and widely applied natural language processing capabilities of LLMs, as well as their emergent abilities, we believe that LLMs like GPT-4o can understand the rules of a card game and process various card-related information. Therefore, we decided to extend the findings and insights from the previous study to turn-based card games. Ultimately, we chose MTG, a popular turn-based card game, as our target game.

2.2. Researchs on MTG

MTG, a collectible card game that has been in continuous operation for over 30 years (1993–2025), boasts a card pool exceeding 90,000 cards. In a standard MTG tournament, players construct a deck containing at least 60 cards (with up to 15 sideboard cards) from the sets currently legal for the season.¹ The standard format typically allows for

a pool of approximately 2500 to 3000 cards, with a maximum of four copies of any single card allowed in a deck.

A previous study highlighted that the complexity and difficulty of MTG make it an excellent platform for testing new agent technologies [24]. However, most research on MTG has primarily focused on deck-building strategies, card selection [25–27], and card design [28], with relatively little work dedicated to developing MTG agents.

Earlier research identified two major challenges in developing MTG agents. First, MTG has highly complex game mechanics. One study specifically analyzed the decision complexity of playing a single card in MTG [29], demonstrating the game's intricate decision-making process. As a result, in a previous study [24], researchers had to narrow their focus to only the combat phase of MTG, using decks that included only creature cards with power and toughness while removing all card effects. The second challenge is that MTG does not have an open-source experimental version. Developing an MTG agent requires implementing the entire MTG system, including all card effects, which poses a significant engineering challenge [24]. The sheer complexity of replicating a commercial game like MTG has led to relatively limited research on MTG agents.

Furthermore, we believe that in order to implement an MTG agent, the agent needs to read and understand cards, analyze card strength, assess the board state, set strategies, and more. These tasks are difficult to accomplish with a single model and require a multimodal approach. Creating such a model is highly challenging. However, the emergence of LLMs may offer new solutions to these challenges, given their powerful natural language processing and reasoning abilities. This paper will explore the use of appropriate prompts and frameworks that enable an LLM to play MTG and implement a DDA mechanism that adjusts its abilities based on the opponent's capabilities.

3. Method

Our approach will extend previous research in two key ways:

- First, we will expand the use of LLMs as a DDA mechanism from a simple dice game [16] to MTG.
- Second, the scope of this study will extend beyond just the combat phase [24]; it will encompass the setup phase, the first main phase, the combat phase, and the second main phase of the game.

The modeling of an LLM as DDA for MTG presented in this paper will be broader than any previous research on the subject. We agree that implementing the entire MTG process is a substantial practical and engineering challenge, so, similar to the previous study [24], we simplified certain aspects of the process. Section 3.1 will show the original rule of MTG, and our simplification will be detailed in Section 3.2.

3.1. Original MTG

In this section, we briefly outline the basic rules of two-player MTG. For the most comprehensive MTG rules, please refer to the official MTG rule book.² Initially, a two-player MTG duel requires two participants, each of whom must prepare their own deck.

At the beginning of the game called setup phase, both players must shuffle their decks and then draw seven cards as their initial hand. If a player is dissatisfied with their starting hand, they may perform an action called “Mulligan”. After taking a mulligan, the player reshuffles their deck and draws a new hand of seven cards. A player may play mulligan multiple times; however, with each mulligan, that player must place M cards from their hand at the bottom of their library (M being the number of mulligans taken). This means that if a player plays

¹ <https://magic.wizards.com/en/how-to-play>.

² <https://magic.wizards.com/en/rules>.

mulligan seven times, this player's initial hand size would be zero; therefore, the maximum allowed number of mulligans is six.

After determining the initial hands and deciding the starting player by a dice roll (the player with the higher roll takes the first turn), the game goes to the play session. In a two-player match, each turn consists of two rounds: the first round belongs to the starting player, and the second to the opponent. Each round is divided into multiple phases, which, in simplified terms, include:

- **Draw Phase** —The active player draws a card.
- **1st Main Phase** —The player can play a land, summon creatures, or cast spells.
- **Combat Phase** —The attacking player declares attackers, followed by the defending player declaring blockers.
- **2nd Main Phase** —The active player can again play a land, summon creatures, or cast spells.
- **End Phase** —The turn ends, and any end-of-turn effects resolve.

This is the MTG structured sequence to ensure a balanced and strategic progression of gameplay.

In a given turn, a player's strategic decisions are largely constrained by their currently available mana resources, as summoning creatures, spells, sorceries, and other cards require sufficient mana to cast. Notably, mana in MTG is divided into six distinct types, and different cards may demand specific combinations of these types. A player's mana pool, both in quantity and variety, is determined by the lands they have played during their main phases. The game features six basic land types, each typically generating one mana of a corresponding type. The lands in play represent the maximum potential mana resources a player can utilize during their turn. Importantly, MTG rules restrict players to playing only one land per turn, further influencing resource management and pacing throughout the game.

Combat in MTG primarily consists of the "Declare Attackers Phase" and the "Declare Blockers Phase" (though players may also cast spells during main phases to deal damage directly to opponents or creatures). During the declare attackers phase, the active player may choose any untapped creatures they control to attack. In particular, attackers cannot target specific entities; by default, all declared attacks are directed at the opposing player unless modified by certain game mechanics. In the declare blockers phase, the defending player may assign any untapped creatures they control as blockers. The defending player must explicitly designate which attacking creature each blocker intercepts. In MTG, the offensive capability of a card is quantified as its power, while its defensive resilience is termed toughness. At the end of the player's turn, any surviving creatures have their toughness fully restored, reflecting the cyclical nature of the game's combat resolution. We believe that this structured combat system emphasizes tactical positioning, resource allocation, and strategic blocking decisions, all of which are fundamental to MTG's gameplay dynamics.

Finally, we need to know that each player begins the game with 20 life points. In a two-player match, the game concludes immediately when either player's life total reaches zero, and the player with a remaining life total greater than zero is declared the winner. Thus, a player's life total serves as a direct and effective indicator of their current in-game health status —the higher the life total, the more favorable their position. For the purposes of our research, we define the game difficulty as the difference in life totals between the human player and the AI opponent. When this difference is positive (i.e., the player's life total exceeds the AI's), the player is considered to be at an advantage, and the magnitude of the difference reflects the ease of the game (larger values indicate simpler gameplay). Conversely, when the difference is negative (i.e., the AI's life total is higher), the player is deemed to be at a disadvantage, with the magnitude correlating to the difficulty of the game (larger absolute values signify greater challenges). This metric provides a quantifiable basis for evaluating dynamic game states and assessing the relative performance of players and AI systems.

3.2. Our game settings of MTG

MTG involves highly intricate rules for each turn, including card interactions and the activation of abilities, making it exceedingly difficult to calculate the decision-making complexity of an entire game. However, previous research has attempted to estimate the decision-making complexity for single-card decisions during individual turns [29]. The aforementioned findings highlight the substantial complexity of MTG and the challenges associated with modeling its gameplay.

Therefore, in the previous research of MTG simplified the whole game system, which focused solely on modeling the combat phase. We aim to go a step further by exploring whether LLMs, after being shown capable of functioning as the DDA in a simplified dice game, can also serve as the DDA for complex commercial card games like MTG. We extend the scope of LLMs to cover MTG's setup phase, the 1st main phase, the combat phase, and the 2nd main phase. Fig. 1 shows our modified MTG game flow. The method for determining the active player in this figure is that both players roll a random 6-sided die, and the player with the higher roll goes first.

Moreover, implementing a complete version of MTG poses a substantial long-term engineering challenge, as nearly every card in the game possesses unique abilities [24]. To address this complexity, this previous work removed all card abilities to simplify the game environment. In our study, we adopt a similar approach, modifying it to suit the objectives and constraints of our experimental design. When a player attempts to activate a card's ability, it is treated as if the card were discarded to the graveyard after paying its cost, with no further impact on the game. However, we retain the option in the prompt to inquire whether the LLM would like to activate a card's ability. This is because we believe that the inclusion of ability cards offers players additional strategic options. Once the complete MTG system is fully implemented, we can future test whether the LLM can play the full game and function as the DDA. To minimize the impact of this limitation in our experiment, we will provide the LLM with the detailed progress of the game, enabling the model to better manage the game flow.

Finally, in a previous study [24], researchers simplified the mana system by removing color distinctions—all mana was treated as colorless. This simplification in previous research meant that the agent did not need to consider mana colors when playing the game, which was a significant reduction in complexity. For example, suppose a player has two land cards and one creature card in hand. One land card generates G (Green) mana, the other generates R (Red) mana, and the creature card requires R mana to be summoned. Under the simplified system, playing any land card would allow the creature to be summoned. However, under the original MTG rules, the creature can only be played if the land card generating R mana is used. In our study, we will not simplify the mana system. Table 1 presents a comparative analysis of three distinct MTG configurations: the original MTG ruleset, our proposed experimental framework, and the modified settings employed in a previous study.

3.2.1. Deck building in MTG

For MTG's deck building, the theoretical number of possible 60-card deck combinations in this format can be roughly calculated using the combinatorial formula $C(N + 59, 59)$, where N is the number of available cards. For $N = 3000$, this results in approximately 10^{108} possible deck configurations. While this number is smaller than the state-space complexity of Go (3^{361} , approximately 10^{172}) [30], it is still exceedingly large. In contrast, for the historic format, where players can use any card ever issued in MTG (with N roughly equal to 90,000), the number of possible 60-card decks increases dramatically to approximately 10^{214} . From a mathematical perspective, this surpasses the complexity of Go's state-space.

Meanwhile, the primary objective of this study is to observe the performance of the LLM as DDA in the game, so the construction of

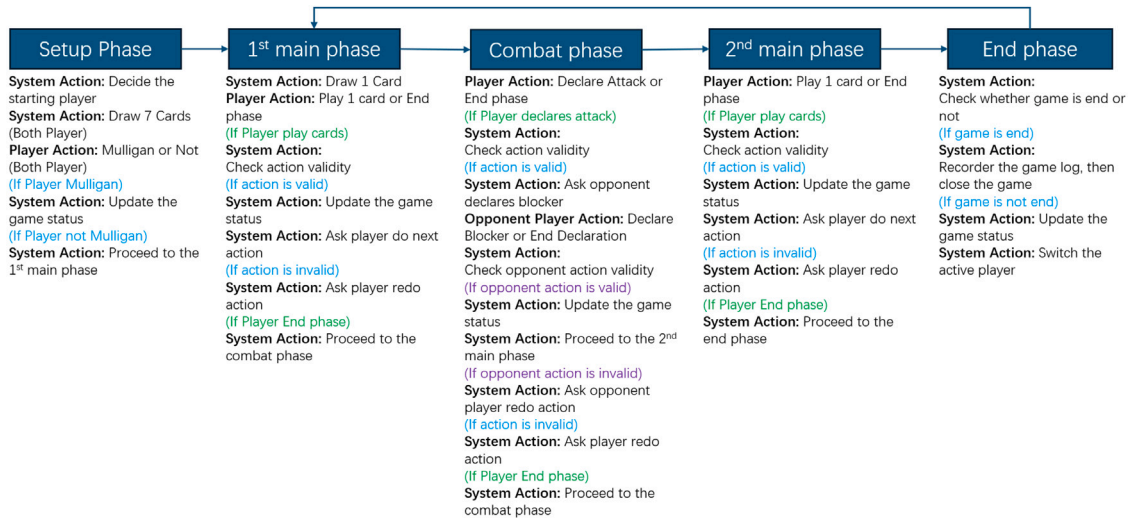


Fig. 1. Our MTG game flow. This figure illustrates all details of our game flow architecture. In this framework, system actions are those determined by the game system without player intervention, whereas player actions require explicit user input and decision-making.

Table 1
Comparison of rule simplifications across settings.

Aspect	Original MTG	Our setting	Cowling et al. [24]
Card abilities	Activated	Removed	Removed
Mana system	Six colors	Six colors	Colorless
Card types	All types	All types	Only creatures and lands
Information	Hidden hands and deck	Different views for different player	Determined via sampling
Turn structure	Full structure	Basic structure retained	Basic structure retained
Interaction	Full Interruption	No human interruptions; Prompt-based action	No in-turn interaction
Agent type	Human players	LLM with prompt	Rule-based and MCTS

Note: Our design keeps key strategic elements like colored mana and full card diversity, while simplifying the combat logic (by removing card abilities) and made the entire battle process automatic for LLM compatibility.

the deck by the LLM is not our focus. Instead, we have chosen to use publicly available deck data created in the MTG draft format. Based on previous research, publicly available deck data created in the draft mode of MTG [25] was obtained. Draft mode is a gameplay format in card games where players do not use their pre-owned cards to build decks. Instead, both players independently construct their decks from a predetermined drafting system.

The drafting system randomly selects three cards from a card pool and presents them to the player, who then chooses one card to add to their deck. The remaining two cards are returned to the card pool. This process is referred to as a draft round. After 60 draft rounds, players will have their decks ready and will compete against other players who have constructed their decks in the same manner. This drafting system is independent for each player, ensuring that all players have access to the same card pool.

We believe that in previous studies, the AI-generated decks in this mode had similar strengths relative to one another. To minimize the influence of deck strength on the outcome of the games, we selected the highest-rated deck from the publicly available draft-mode deck data in the previous research. Both players utilize this identical deck during gameplay, thereby controlling for extraneous variables and ensuring that deck variations do not confound the experimental results. The deck data we selected is publicly available online.³

3.2.2. Two players

In this experiment, we believe that to effectively test the LLM's ability to function as the DDA for MTG (LLM-MTG-DDA), its opponent

could also be an LLM. This design decision is motivated by two key considerations. First, the use of human players introduces subjective variability that may confound the evaluation. Specifically, the limitations imposed on card ability activations—necessary for the DDA's operation—could alter human players' strategic behaviors in unpredictable ways. Such changes may introduce biases or unintended shifts in play style, thereby complicating the assessment of the DDA's performance. In contrast, employing another LLM as the opponent (hereafter referred to as LLM-Player) enables a more controlled environment. The LLM-Player is not designed to perform DDA tasks; its sole objective is to win the game, simulating the behavior of a standard, competitive player. We further assume that the LLM-Player is relatively insensitive to the altered rules, and that its strategies are more consistent and objective than those of human participants under the same conditions. Second, the use of LLMs significantly reduces the cost and complexity associated with large-scale experimentation. While human-based evaluation demands considerable time, coordination, and resources, employing LLMs as opponents allows for broader and more efficient experimentation across diverse gameplay scenarios. Therefore, in our experimental setup, both players are instantiated as LLMs (GPT-4o). Player 1, the LLM-Player, is focused on defeating its opponent. Player 2, the LLM-MTG-DDA, acts as the DDA agent, aiming to dynamically balance the game experience.

3.2.3. Player view setting

According to the MTG game scenario shown in Fig. 2, each player has their own hand, library, graveyard, battlefield, and life points. For player 1 (LLM-Player), we provide a player view. The LLM can only access information about their own hand, library, graveyard, battlefield, and life points, as well as detailed information about Player 2's battlefield, graveyard, and life points, along with the number of

³ <https://drive.google.com/drive/folders/1kFqktfFU2oKcsTh3wLb500ozy8IqxMt7?usp=sharing>.

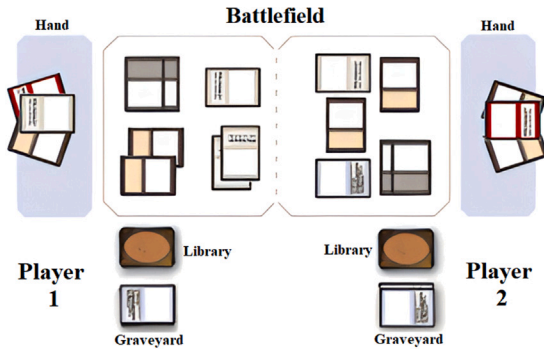


Fig. 2. MTG game layout.

cards in Player 2's hand and library. Conversely, we provide a creator view for player 2 (LLM-DDA), which is all information on the game field.

The main reason for this setup is that the objectives of the LLM-Player and LLM-DDA are entirely different. The LLM-Player aims to emulate the experience of a human player, so we provide it with information from the player's view. In contrast, the goal of the LLM-DDA is to balance the difficulty of the game and act as the game's DDA. Therefore, we give the LLM-DDA a creator view to enable better control over the game's difficulty and to reduce the impact of randomness in the card arrangement following shuffling, which could otherwise affect the LLM-DDA's ability to manage the game's difficulty. In subsequent phases, we will also grant the LLM-DDA certain privileges to make its control over the game difficulty more manageable.

The card information provided to the LLM includes not only the card names but also the current state of the cards, which encompasses the name, cost, card description, power, toughness, and whether the card is tapped. We believe this information is crucial, as it informs the LLM whether it can attack, whether it can defend, and whether it has sufficient resources to play the desired cards, among other critical details derived from the game state. We expect that this information will assist the LLM in analyzing its next move and determining whether it is in a favorable or unfavorable position.

3.3. Prompt design

We argue that prompt design plays a critical role in our framework. Not only do prompts vary depending on the specific LLM objectives, but they also require dynamic adaptation in response to in-game changes. Fig. 3 illustrates the overall system workflow of our approach.

Furthermore, in the subsequent section, we systematically analyze our prompt designs and their respective functional impacts.

Section 3.3.1 delineates the design and implementation of Prompt 1, Prompt 2.1, and Prompt 2.2. Section 3.3.3 presents the core interaction prompt (Prompt 3) deployed during the main phase of gameplay. Section 3.3.4 details our combat scenario-specific prompts, namely Prompt 4.1 and Prompt 4.2, which govern battle mechanics. Finally, Section 3.3.5 provides a systematic analysis of our framework's handling of invalid actions.

3.3.1. A unified system prompt for all phases

A system prompt, shown in Prompt 1, is standardized to ensure consistency in the LLM's performance throughout the game, including aspects such as objectives. In the system prompt, *obj_txt* represents the objectives of the two players controlled by a selected LLM, each having an independent system prompt. The user prompt will reiterate *obj_txt* to emphasize their goals and responsibilities before the LLM makes decisions.

Prompt 1: System Prompt for Both Players

You are a Magic: The Gathering player engaged in a game with an opponent. You already own a deck. You start with 20 life points.

Magic: The Gathering Rules for you:

Your Objective: *obj_txt*

The game flow of Magic: The Gathering includes the following phases:

The setup phase, which is executed only once at the initial of the game, allows you to change your hand as needed.

Each turn consists of the Draw Phase, Main Phase, Combat Phase, Opponent's Declare Blockers Step, and Ending Phase.

Remember to play Land in each round, if you can.

Critical Term Explanation:

Mana: To generate mana, you must TAP a land. Lands produce a color of mana indicated on the card. Land

cards are not spells. You may play one during each of your turns for free.

Creatures: Creature cards can attack opponents or block their attacks.

Spells: Include instants, sorceries, enchantments, etc.

Abilities: Creatures and other permanents may have special abilities.

3.3.2. User prompt for setup phase

The setup phase in MTG is a unique phase that occurs only once at the very beginning of the game. At the start, each player draw seven cards. If a player is not satisfied with their hand, they may redraw seven cards in an action called a "MULLIGAN". However, for each mulligan taken (denoted as x), the player must choose x cards from the redrawn hand of seven cards to put back into the deck. This phase reflects a distinctive rule of MTG.

We designed different prompts for the LLM-Players and LLM-DDA during this phase. Prompt 2.1 is for the LLM-Player during the setup phase. Prompt 2.2 is for the LLM-DDA during the same phase.

Prompt 2.1: Setup Phase for LLM-Player

You need to determine your actions based on the Phase Explanation and Critical Term Explanation.

Currently, it is the Setup Phase.

Do you want to MULLIGAN?

Please consider your next action step by step, taking into account the strength of each card, the mana cost required, the special abilities of each card, and your own health.

Additionally, you should respond using the following format: /Your Answer : (YES or NO)

Phase Explanation: In this phase, your HAND will have seven cards from your LIBRARY. If a you are unhappy with your hand, they may take a MULLIGAN.

Critical Term Explanation: To MULLIGAN, shuffle your hand into your LIBRARY and draw seven cards. Then, put a card of your choice on the bottom of your LIBRARY for each time you MULLIGANED this game.

Current Situation: *Current_Situation_P1*

Invalid_Action

Prompt 2.2: Setup Phase for LLM-DDA

You need to determine your actions based on the following statement.

Currently, it is the Setup Phase.

Please select between 5 and 7 cards from your library to form your hand.

Please consider your next move step by step, taking into account the strength of each card, the mana cost required, the special abilities of each card, and your own health.

In addition, you should respond using the following format /Cards: Card-Name1, CardName2, ...

Current Situation: *Current_Situation_P2*

Invalid_Action

The differences between prompt 2.1 and prompt 2.2 reveal distinct roles in the setup phase. Prompt 2.1 guides the LLM-Player to follow the MTG setup phase rules, while prompt 2.2 grants the LLM-DDA special privileges. The LLM-DDA is not required to adhere to the mulligan rules and can instead select its desired starting hand directly from the

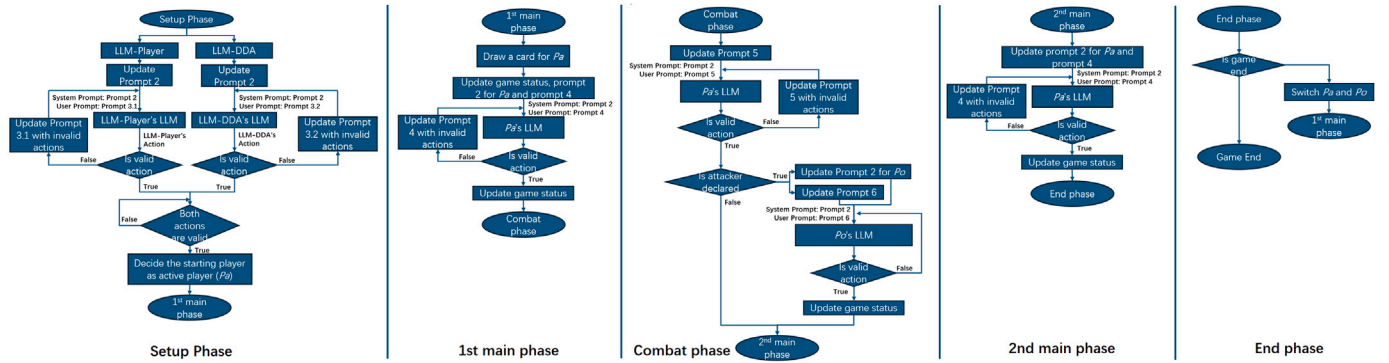


Fig. 3. Our system flow (P_a represents the active player, and P_o represents P_a 's opponent).

deck. This adjustment aims to give the LLM-DDA better control over the game's difficulty, aligning with the rationale for providing different perspectives, as discussed in Section 3.2.3.

In the prompts, *Current_Situation_Pi* represents two players' view described in Section 3.2.3. Here, *Current_Situation_P1* corresponds to the player view, while *Current_Situation_P2* corresponds to the creator view. However, in the creator view, we do not input the entirety of the LLM-Player and LLM-DDA libraries into the prompt, as this would make the prompt excessively long and hinder the LLM's performance. Instead, the creator view provides information on the top three cards of both the LLM-Player's and LLM-DDA's libraries. In addition, at the end of each prompt, we have included a line to alert the player if invalid actions have been detected. This will be detailed in Section 3.3.5.

3.3.3. User prompt for 1st main phase and 2nd main phase

In MTG, the two main phases of a player's turn are typically the only times when non-instant spells, such as artifacts, creatures, enchantments, planeswalkers, battles, and sorceries, can be cast. These phases are also when players can normally play a land. Since the actions available to players during these two phases are identical, we use the same prompt to manage both phases.

If a player has already played a land during the 1st main phase, the prompt for the 2nd main phase of that turn will include a note in the *Invalid_Action* line with, "You have played a Land this turn". Prompt 3 illustrates the prompt designed for the main phases.

Prompt 3: Main Phase (1st and 2nd) for Both Players

Based on the Current Situation, please choose one of the following actions:
 Action 1: Play a card from your hand that aligns with your game objectives (this step may cost mana).
 Action 2: Activate an ability of a card or cast a spell card (this step may cost mana).
 Action 3: End this phase.
 You need to carefully consider which action best aligns with your game objectives. Please think it step by step.
 Play one card only. We will ask you if you are willing to play another card in the next time.
 Objectives: *obj.txt*
 If you choose Action 1, please respond in the following format: /[1.Name one card you want to play]
 If you choose Action 2, please respond in the following format: /[2.Name that card (a Instant or Sorcery), your target(a card name or self or opponent), target owner(self or opponent)]
 If you choose Action 3, please respond in the following format: /[3.End This Phase]
 Current Situation: *Current_Situation_Pi*
 Invalid_Action

During the 1st main phase and 2nd main phase, the primary differences in the prompts received by the LLM-Player and LLM-DDA are *obj.txt* and *Current_Situation_Pi*, which we have explained extensively

in previous sections. In the prompt, we restrict the LLM to playing only one card at a time. Whenever the LLM successfully plays a card, resulting in changes to the game state, we immediately update the current situation to ensure that all subsequent decisions are based on the most up-to-date information.

Additionally, as long as the LLM selects either Action 1 or Action 2, the system will repeatedly input the updated prompt to the LLM until it selects Action 3 to end the phase. However, to prevent the LLM from getting stuck in an infinite decision loop, we have implemented a mechanism in the *Invalid_Action* line to forcibly end the current phase if necessary. Detailed information about this mechanism can be found in Section 3.3.5.

3.3.4. User prompt for combat phase and declare blockers phase

In MTG, the combat phase is divided into two stages: the active player's declare attackers stage and the opponent's declare blockers stage. The latter is only triggered if the former occurs. For simplicity, we have engineered these stages as the combat phase and declare blockers phase.

During the combat phase, the declare blockers phase is only triggered if the active LLM selects Action 1: Declare Attackers. Similar to the main phase, the game proceeds to the next phase only when the active LLM selects Action 3: End This Phase or when the mechanism for forcibly ending the current phase is triggered.

In the declare blockers phase, the attackers declared by the active LLM during the combat phase are included in the prompt for the opponent LLM. This information allows the opponent to decide whether to defend and, if so, which of their cards to use as blockers against specific attacking cards. Prompts 4.1 and 4.2 provide a detailed explanation of the mechanisms involved in these phases.

Prompt 4.1: Combat Phase for Both Players

Based on the current situation, please choose one of the following actions:
 Action 1: Declare one attacker.
 Action 2: Activate an ability of a card (this step may cost mana).
 Action 3: End this phase.
 You need to carefully consider which action best aligns with your game objectives. Please think it step by step.
 Play one card only. We will ask you if you are willing to play another card in the next time.
 Objectives: *obj.txt*
 If you choose Action 1, please respond in the following format: /[1.Name one card you want to attack]
 If you choose Action 2, please respond in the following format: /[2.Name one card (a Instant or Sorcery), your target(a card name or self or opponent), target owner(self or opponent)]
 If you choose Action 3, please respond in the following format: /[3.End This Phase]
 Current Situation: *Current_Situation_Pi*
 Invalid_Action

Prompt 4.2: Declare Blockers Phase for Both Players

Based on the current situation, please choose one of the following actions:
 Action 1: Declare one blocker.
 Action 2: Activate an ability of a valid card (this step may cost mana).
 Action 3: End this Step.
 You need to carefully consider which action best aligns with your game objectives. Please think it step by step.
 Choose one card only. We will ask you if you are willing to play another card in the next time.
 Objectives: *obj_txt*
 If you choose Action 1, you need to specify which card you will use to block each of your opponent's cards.: */[1.One Blocker's name, One Attack's name]*

 If you choose Action 2, please respond in the following format: */[2.Name one card (a Instant or Sorcery), your target(a card name or self or opponent), target owner(self or opponent)]*
 If you choose Action 3, please respond in the following format: */[3.End This Phase]*
 Current Situation: *Current_Situation_Pi*
 Being Attacked by: *Opponent_Declared_Attackers*
Invalid_Action

3.3.5. Invalid actions

In each phase of the game, we monitor the LLM's action to detect any invalid actions, such as attempting to play a card without sufficient mana or trying to play a card not present in its hand. We have implemented detailed rules within the system to detect these invalid actions. When an invalid action is identified, it is recorded as a line in *Invalid_Action* of the prompt. The prompt is then re-inputted to the LLM with *Invalid_Action*, helping it to make a new decision.

It is worth noting that during our testing, we observed that the LLM might repeat the same invalid action multiple times, occasionally leading to a deadlock. To address this, we implemented a rule where if an LLM accumulates more than 20 lines of *Invalid_Action* within a single phase, the system forcibly ends the current phase. This design was inspired by a similar mechanism in real-world gameplay, where a player's phase is automatically ended if that player failed to make a valid move within a certain time limit.

4. Evaluation metrics

When evaluating the effectiveness of a DDA system without human participants, analyzing game log data is usually sufficient. However, since the DDA in this study is controlled by an LLM, assessing its difficulty adjustment solely through game logs remains inconclusive. This is because there is currently no research that clearly demonstrates how LLMs understand the game difficulty. Therefore, this study will also analyze the game's difficulty curve to better reveal the LLM's understanding of the game and the appropriateness of its difficulty adjustments.

Section 4.1 will explain our methodology and approach for analyzing game logs in detail. Section 4.2 will introduce a new evaluation metric for assessing LLM-based DDA: the difficulty arc. We base our approach on a previous study [16] that publicly shared data on GPT-4o as a game DDA. In that study, researchers used both human feedback and data augmentation techniques to score different prompts based on fun, challenge, and relaxation. By analyzing the log data from that study and constructing a difficulty arc, we aim to identify the key features of a difficulty arc that make a prompt suitable for DDA. This will serve as a valuable benchmark for evaluating our LLM-MTG-DDA system.

4.1. Game log

In analyzing the game data, we follow the methodology from previous research [16]. First, we examine whether the LLM-DDA opponent, called LLM-Player, has an overall win rate and a win rate per round that are close to 50%. In the previous study [16], the win rate of each round was determined by the dice roll.

In our simplified version of MTG, the game's outcome is determined by the players' health points. Therefore, the win rate per round depends on the difference in health between the two players. If, in a given round, the LLM-Player has more health than the LLM-DDA, we consider the LLM-Player to be at an advantage and define this as a win for the LLM-Player in that round. Additionally, we define a round as the period from when either player starts their turn to when they end their turn.

4.2. Difficulty arc

GPT-4o has been shown to manipulate its dice rolls in a simple dice game using appropriately designed prompts, thereby influencing the opponent player's win rate in the previous research [16]. This suggests that GPT-4o can serve as an effective tool for DDA in turn-based simple games. However, the specific mechanisms by which different prompts control the difficulty of the game warrant thorough investigation. Understanding these mechanisms is crucial, because revealing the details of how prompts affect game difficulty can assist game designers in fine-tuning difficulty curves in games controlled by LLMs. This, in turn, improves our understanding of LLMs and improves our ability to control them. Such insights could significantly enhance the controllability of LLM-based systems.

Additionally, analyzing the difficulty curves shaped by different prompts provides insight into the types of game curves that players find engaging. Identified curves that make games enjoyable for players could serve as a valuable benchmark for evaluating DDA models. Also, understanding the features of curves that contribute to player enjoyment will be a central focus of our analysis. These findings could play a key role in guiding future prompt design and optimizing.

Based on the aforementioned reasons, it is necessary to plot the difficulty curves of the game controlled by GPT-4o under different prompts in the previous study [16]. In this study, we named the concept of game difficulty curves "difficulty arc". This concept is derived from and builds upon a framework established in previous research. In a previous study [31], the authors refined Costikyan's definition of a play arc [32], describing it as a time series of significant in-game events that construct the narrative in the game. Inspired by this concept, our difficulty arc in this study refers to the trajectory of a parameter representing game difficulty throughout the course of a single game.

Building on this concept, we further propose manually defining certain features within the difficulty arcs, including sustained disadvantage phases, sustained advantage phases, stalemate phases, reversal phases, being-reversed phases, and others. These defined phases aim to distinguish between different types of arcs and identify features associated with positive player experiences. It is important to note that the definitions of these features are inherently game-specific, as concepts such as "stalemate" or "reversal" can vary between games. Therefore, the manually defined features must be appropriately adjusted based on the features of the game in question. Below, we outline the definitions of each feature:

- **Sustained disadvantage phase:** This refers to a phase in which the difficulty arc remains consistently below a threshold T for i consecutive turns, indicating that the current game state is advantageous for the opponent.
- **Sustained advantage phase:** This refers to a phase in which the difficulty arc remains consistently above a threshold T for i consecutive turns, indicating that the current game state is disadvantageous for the opponent.

- **Stalemate phase:** This refers to a phase where the difficulty arc fluctuates within a range of $\pm j$ for i consecutive turns, suggesting that neither the current player nor their opponent can effectively shift the game's balance during this period.
- **Reversal phase:** This occurs when, after a sustained disadvantage phase, the difficulty arc shifts, placing the player in an advantageous position with i turns or leading directly to a win. (This is typically represented as a “v” or “u”-shaped pattern in the difficulty arc).
- **Being-reversed phase:** This occurs when, after a sustained advantage phase, the difficulty arc shifts, placing the player in a disadvantageous position with i turns or leading directly to a loss. (This is typically represented as an inverted “v” or “u”-shaped pattern in the difficulty arc).
- **Win with a reversal at the end:** In the final i turns of the game, the difficulty arc briefly falls below T , but eventually exceeds T and results in victory.
- **Lose with a being-reversed at the end:** In the final i of the game, the difficulty arc briefly falls above T , but eventually drops below T and results in failure.
- **Others:** Any phases not falling under the categories described above are classified as others.

Here, the parameters T , i , j , and are flexible and should be adjusted according to the specific requirements and mechanics of the game.

Based on our defined difficulty arc, users can adapt it to their own games by adjusting parameter values across different features. In our framework, the x -axis of the difficulty arc represents the number of rounds, while the y -axis corresponds to the health-point difference between players. Users have the flexibility to assign custom parameters to both axes to ensure the difficulty arc aligns with their specific game design requirements. Finally, when analyzing the features within the difficulty arc, users can independently determine which features most significantly influence game difficulty based on their own criteria.

4.2.1. The difficulty arc shown by GPT-4o with different prompts

In this section, we present the difficulty arcs of prompts deemed highly significant in the context of the dice game in the previous research [16]. Thanks to the publicly available data from previous research [16],⁴ we can conduct a detailed analysis of the difficulty arc under different prompt controls. The publicly available dataset includes a total of 11 different prompts, with each prompt having 100 game sessions. The feature encompassed within these difficulty arcs are analyzed. The results of this analysis aim to assist game designers in better understanding the features of game difficulty when LLMs are utilized as tools for DDA.

Additionally, the analysis seeks to identify which difficulty features within these arcs contribute to an enhanced player experience. Where appropriate, we propose that these features could serve as one of the metrics for evaluating the “fun factor” of a game. With these reasons, the selected plays controlled by prompts in the previous study [16] include:

- **50:** The prompt yields the best player experience.
- **Hard:** The prompt results in the worst player experience.
- **Easy:** The prompt with the lowest difficulty.
- **0:** The prompt with the highest difficulty.
- **NoGoal and NoGoalCoT:** Prompts reflecting the original tendencies of GPT-4o without specific goals.

The difficulty arc represents the curve that illustrates the progression of the difficulty of the game. In the previous study [16], the parameter used to represent the difficulty of the game is the difference in health points between the two players. Player 1 is considered a proxy

for a human player, while Player 2, controlled by the LLM, represents the AI opponent that a human player might encounter in a real-world scenario. Thus, for Player 1, the game difficulty is reflected in the curve of Player 1's health points minus Player 2's health points over the course of the game.

To construct this curve, we record the health point difference between Player 1 and Player 2 at every turn and normalize the game's total turns to a range of 0 to 1. Here, 0 corresponds to the first turn of the game (that is, 0% of the completed game), and 1 corresponds to the final turn (that is, 100% of the completed game). This normalization allows us to obtain a standardized difficulty arc for each game. Subsequently, for each prompt, we calculate the average difficulty arc across 100 previous game logs, yielding a representative difficulty arc for that specific prompt.

Based on our understanding of the game from previous research [16] and the definition of features in the difficulty arc, we set $T = 0$, $i = 20\%$ of the game's progress, and $j = 0.5$. Based on these parameters, the precise definitions of the features in a difficulty arc are as follows:

- **Sustained disadvantage phase (Player 1):** Over a continuous 20% segment of the game, the difference between Player 1's health points and Player 2's health points remains less than 0.
- **Sustained advantage phase (Player 1):** Over a continuous 20% segment of the game, the difference between Player 1's health points and Player 2's health points remains greater than 0.
- **Stalemate phase (Player 1):** Over a continuous 20% segment of the game, fluctuations in the difficulty arc remain within ± 0.5 .
- **Reversal phase (Player 1):** After being in a sustained disadvantage phase, Player 1's difficulty arc's value greater than 0 for at least 20% of the game or results in Player 1 achieving victory.
- **Being-reversed phase (Player 1):** After being in a sustained advantage phase, Player 1's difficulty arc's value less than 0 for at least 20% of the game or results in Player 1 losing the game.
- **Win with a reversal at the end (Player 1):** In the final 20% turns of the game, the difficulty arc briefly falls below 0, but eventually exceeds 0 and results in victory.
- **Lose with a being-reversed at the end (Player 1):** In the final 20% of the game, the difficulty arc briefly falls above 0, but eventually drops below 0 and results in failure.
- **Others (Player 1):** Any phases not falling under the categories described above are classified as others.

We first examine the difficulty arc of instruction 50 and instruction Hard (Fig. 4). The difficulty arc for instruction 50 includes stalemate phase, sustained advantage phase, being-reversed phase, reversal phase, and ultimately winning with a reversal at the end. In contrast, the difficulty arc for instruction Hard includes stalemate phase, sustained disadvantage phase, reversal phase, being-reversed phase, and ultimately losing with a being-reversed at the end. From Fig. 4, we observe two major differences between the two cases:

- Instruction 50 features a reversal leading to victory, while instruction Hard involves being reversed and ultimately losing the game.
- Instruction 50 has a sustained advantage phase, whereas instruction Hard exhibits a sustained disadvantage phase.

Here, we present the difficulty arcs of the prompts instruction Easy and instruction 0 (Fig. 5). The difficulty arc for instruction Easy includes stalemate phase and sustained advantage phase, while the difficulty arc for instruction 0 includes stalemate phase and sustained disadvantage phase. From Fig. 5, we observe a key difference that instruction Easy consistently remains in a sustained advantage phase, whereas instruction 0 consistently remains in a sustained disadvantage phase.

Finally, we present the difficulty arcs of the prompts instruction NoGoal and instruction NoGoalCoT (Fig. 6). The difficulty arc for

⁴ <https://github.com/Lxx007/LI-LLM-DDA-UX/tree/main>.

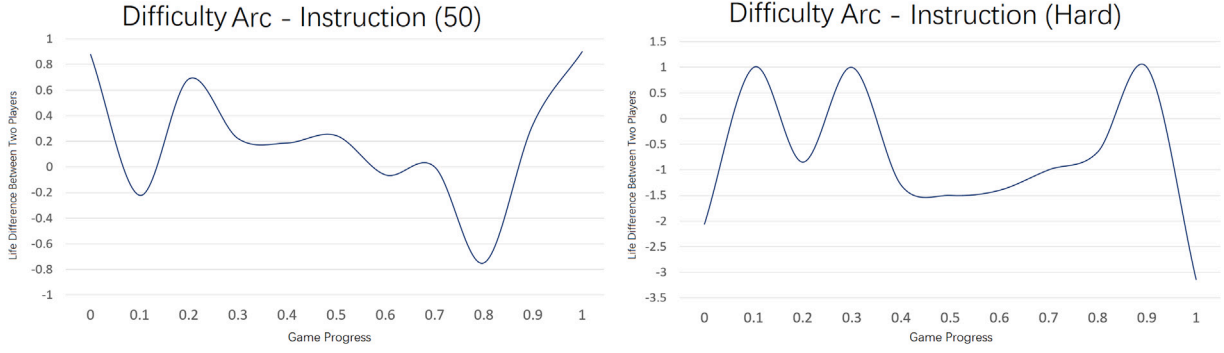


Fig. 4. Difficulty arc of instruction 50 and Hard.

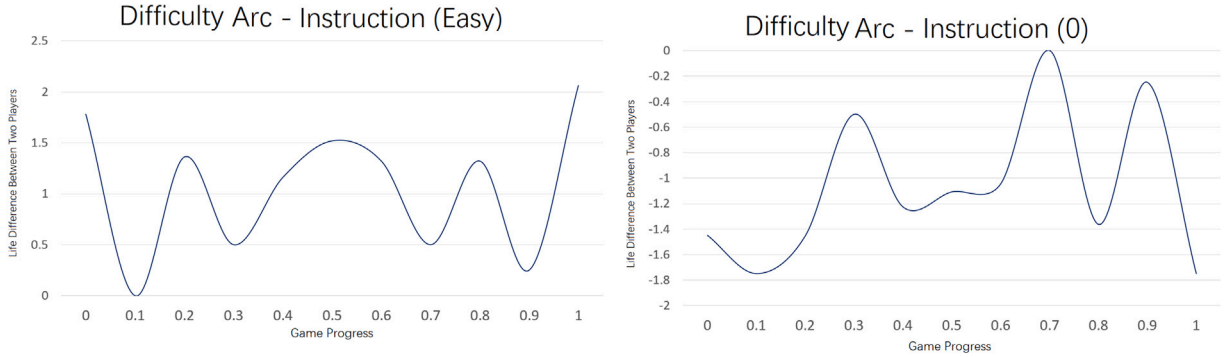


Fig. 5. Difficulty arc of instruction Easy and 0.

instruction *NoGoal* includes a stalemate phase and a sustained disadvantage phase, while the difficulty arc for instruction *NoGoalCoT* includes stalemate phase, sustained disadvantage phase, and losing with a being-reversed phase at the end. From Fig. 6, we observe that the player spends most of the game in a disadvantageous position, and in the case of *NoGoalCoT*, a reversal occurs that leads to a loss at the end. If the parameter i is slightly extended to 25%, instruction *NoGoal* would also result in a situation of being reversed and result in loss.

From the difficulty arcs presented above, we can observe the following situations. First, reversal victories occur only in the prompt associated with the best player experiences. Second, reversal defeats occur in prompts associated with poor player experiences. Lastly, keeping players consistently in either an advantageous or disadvantageous position leads to a game that is either too easy or too difficult. Therefore, the ideal DDA difficulty arc should encompass the following three elements:

- The game is better for player reversal victories.
- The game does not allow for player reversal defeats.
- The game should not keep the player consistently in an advantageous or disadvantageous position.

5. Experiment

Our game environment is fully simulated through a Python-based application. The LLM employed in this study is GPT-4o, which is accessed via API calls. Both the prompts and the complete game environment source code have been made publicly available on an open-access repository.⁵

For our LLM-MTG-DDA framework, one objective within the prompt is the primary means of controlling their game goals.

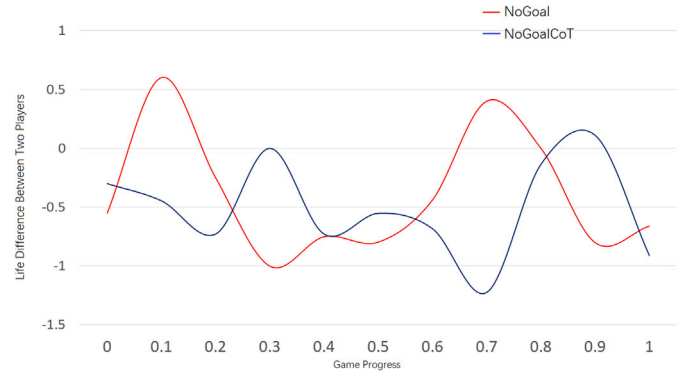


Fig. 6. Difficulty arc of instruction NoGoal and NoGoalCoT.

- For Player 1 (LLM-Player), the *obj_txt* is “Reduce your opponent’s life points from 20 to 0”.
- For Player 2 (LLM-DDA), we adopt the conclusions from the previous study [16], the *obj_txt* is “Make your opponent’s round win rate in 50 percent. Here is the historical plays. Your opponent life: *Player_life_S*; Your life: *DDA_Life_S*”.

Player_life_S and *DDA_Life_S* represent the historical life sequences of the LLM-Player and LLM-DDA, respectively, in the current game.

However, in a previous study [16], it is not indicated that such a prompt design would be applicable to all turn-based games. The key insight from that study is that prompt designs like their design may be effective. Based on this idea, we created our own objective based on our own understanding on MTG for the LLM-DDA on balancing the life points. Simultaneously, our goal aims to empower the LLM to perform dynamic difficulty adjustments. We need to identify an optimized prompt formulation capable of effectively guiding the

⁵ <https://drive.google.com/drive/folders/1kFqktfFU2oKcsTh3wLb500ozy8I xqMt7?usp=sharing>.

Table 2
Details of three groups' datasets.

	Group 1	Group 2	Group 3 (Baseline)
LLM-Player's overall win rate	0.70	0.58	0.45
LLM-Player's round win rate	0.39	0.28	0.24
LLM-Player's round lose rate	0.21	0.25	0.31
LLM-Player's round draw rate	0.40	0.47	0.45
LLM-Player's round win rate[No Draws]	0.65	0.53	0.44
Number of rounds per game (AVG)	18.03	18.38	17.01

LLM's behavior. To systematically evaluate this, we conducted three experimental groups.

In groups 1 and 2, the LLM-DDA (opponent agent) employed distinct prompt strategies—one utilizing established prompts from the previous research [16], and another implementing our novel prompt design derived from that literature. Group 3 served as the baseline, where the LLM-DDA maintained identical objectives to the LLM-Player. In this case, the LLM-DDA would no longer have the DDA goal, but would function as a privileged LLM-Player. This control group enables a clearer differentiation between the effects observed in groups 1 and 2. The following shows three sets of groups:

- **Group 1:** LLM-Player vs. LLM-DDA with *obj_txt*: “Make your opponent's round win rate in 50 percent. Here is the historical plays. Your opponent life: *Player_life_S*; Your life: *DDA_Life_S*”.
- **Group 2:** LLM-Player vs. LLM-DDA with *obj_txt*: “Balance your life points with your opponent's. Aim to make the difference as close to zero as possible”.
- **Group 3 (Baseline):** LLM-Player vs. LLM-DDA with *obj_txt*: “Reduce your opponent's life points from 20 to 0”.

Both the LLM-Player and LLM-DDA in our experiment are implemented using GPT-4o via its API, as it represents the current SOTA in LLMs. For each group, we conduct 100 games, recording the life points of Player 1 (LLM-Player) and Player 2 (LLM-DDA) at each turn. The difficulty arc of the game is represented by the trend in the difference between the LLM-Player's life points and the LLM-DDA's life points. The specific algorithm used to calculate the difficulty arc follows the method outlined in Section 4.2.1.

6. Results and discussions

6.1. Game log

Table 2 presents the difficulty details of the three experimental groups, including the overall win rate of the LLM-Player, the round win rate of the LLM-Player, and the average number of game rounds for each group. The round win rate of the LLM-Player is calculated based on the difference between the LLM-Player's life points and the LLM-DDA's life points in each round. A negative value indicates a loss for the LLM-Player, a positive value indicates a win, and a value of zero indicates a draw for that turn.

The baseline overall win rate is already close to 50%, suggesting that, in terms of overall win rate, the prompts used for the two DDA configurations are not better than the baseline. Notably, Group 2's overall win rate is closer to 50% than that of Group 1. However, the overall win rate of the LLM-Player does not fully reflect the effectiveness of the LLM-DDA in managing the difficulty on a turn-based game. From the round win rate of the LLM-Player, we observe more interesting patterns.

First, Group 2 has the highest number of draws, with the LLM-Player's round win rate and round loss rate in each round being approximately equal. Excluding draws, the LLM-Player's round win rate is 52.929% in Group 2, 64.537% in Group 1, and 44.067% in Group 3. Group 2 clearly demonstrates a more reasonable DDA compared to the other two groups. Furthermore, regarding the average number of

rounds in the games, Group 2 also has the highest value, indicating that the LLM-DDA in Group 2 is better able to adapt to the player, which is superior than our baseline. In contrast, Group 1, which uses a prompt designed to perform well in the previous research [16] is far less effective in enabling the LLM to act as a DDA, even performing worse than our baseline, except for the value of the average number of rounds in all games.

6.2. Difficulty arc

After analyzing the game log data, we now examine the difficulty arcs of the three groups. Fig. 7 presents the difficulty arcs for the three groups. From this figure, we observe that the difficulty arc created by the LLM-DDA in Group 1 for the LLM-Player includes a stalemate phase and a sustained advantage phase, with the LLM-Player consistently maintaining an advantage. The difficulty arc for Group 2 includes a stalemate phase, a sustained disadvantage phase, and a win with a reversal at the end, indicating a shift in difficulty.

For Group 3, the difficulty arc for the LLM-Player is essentially the reverse of Group 1's, consisting of a stalemate phase and a sustained disadvantage phase, with the LLM-Player consistently at a disadvantage.

At the end of Section 4.2.1, we mentioned three key metrics for the difficulty arc:

- The game is better for player reversal victories.
- The game does not allow for player reversal defeats.
- The game should not keep the player consistently in an advantageous or disadvantageous position.

The only LLM-DDA that satisfies all three of the above conditions is the one from Group 2. Based on the comprehensive game log data and difficulty arc, for our simplified version of MTG, the prompt with “Balance your life points with your opponent's. Aim to make the difference as close to zero as possible”. proves to be a successful LLM-MTG-DDA.

We posit that adhering to Csikszentmihalyi's flow theory [33] in game design generally enhances player enjoyment. Sustained player dominance or disadvantage fails to maintain long-term engagement, necessitating balanced win/loss outcomes between players and AI systems. This theoretically validates our analytical approach to difficulty arcs and DDA. Furthermore, we believe that in-game reversals of fortune elicit significant emotional arousal, with positive-outcome reversals particularly enhancing player affect. Although empirical research in this domain remains limited, our design paradigm for MTG AI opponents implicitly cultivates beneficial emotional states through calibrated reversal mechanics.

From the above results, we can observe that using the difficulty arc to assess the rationality of a game's DDA is a reliable approach. Additionally, the round win rate and the average number of rounds in a game should also be important criteria for evaluating the reasonableness of a DDA. However, an unexpected finding in our results was that the prompt with “Make your opponent's round win rate in 50 percent”. along with historical battle data did not effectively function as a DDA in our simplified MTG.

In contrast, using the prompt to balance the life points of the players and bring the difference close to zero proved to be an effective way of guiding the LLM to become a DDA in MTG. This highlights that, as the complexity and rules of the game change, the prompt must be more closely aligned with the game, and there is not a universal prompt that can make an LLM achieve DDA in all games. We speculate that in a complex game like MTG, controlling the round win rate of players through LLM is highly challenging, while balancing the difference in life points between the players is more feasible. Therefore, we believe that when using an LLM as a DDA, the feasibility of the balancing task in the game should be considered. The higher the feasibility of the goal provided in the prompt, the greater the success rate of the LLM.

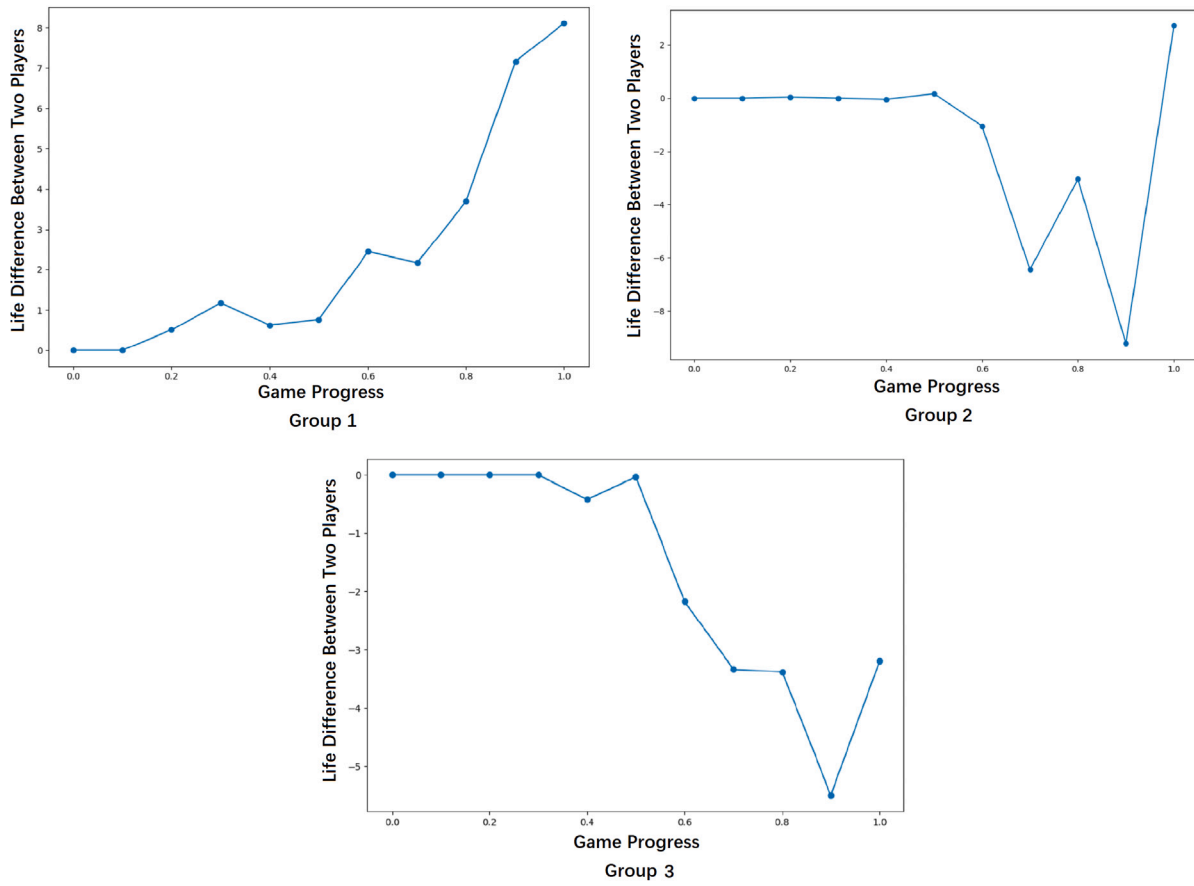


Fig. 7. Three group's difficulty arc controlled by corresponding LLM-DDA for their LLM-Player.

Moreover, we were pleased to discover that GPT-4o is capable of understanding and playing our simplified version of MTG. Moreover, we boldly speculate that GPT-4o, with the appropriate prompt, could understand and play the full version of MTG. This is based on our observations of GPT-4o's CoT responses while playing the simplified MTG. It was able to identify the abilities of creatures and planeswalkers on the field and accurately describe their threats. In addition, GPT-4o could understand the descriptions of specialized MTG terms such as tap, defender, reach, and deathtouch, and make decisions based on these descriptions.

Finally, in the game, lands produce mana of the corresponding color, and playing cards requires mana of specific colors. We were surprised to find, from GPT-4o's CoT responses, that it was strategically planning its mana and selecting the appropriate lands to summon target cards, rather than randomly playing lands.

7. Conclusions

Based on our results, we can conclude that using our LLM-MTG-DDA framework in our simplified version of MTG is highly effective. With an appropriate prompt, the LLM can balance the player's win rate close to 50%, and excluding draw plays, the round win rate remains approximately 50%. We believe that designing an LLM-MTG-DDA framework must take into account the inherent complexity of the game. The game should be divided into smaller, independent segments where decisions can be made separately, allowing the LLM to determine its next action based on the current game state. This approach is necessary because using a single, large prompt to control the entire game would result in excessively long prompts, making it difficult for the LLM to comprehend and make informed decisions. Additionally, a

modular framework like this could be more adaptable to various card games, providing insights for applying LLM-MTG-DDA to other similar games, such as Hearthstone or Yu-Gi-Oh!

Furthermore, in our designed framework, we allow the LLM to play only one card at a time. This decision is based on three main reasons:

- **Better Control Over the LLM**—Limiting the LLM to playing one card at a time makes it easier to manage and guide its actions.
- **Consistency with Human Gameplay**—In real gameplay, players also play cards one by one rather than playing multiple cards simultaneously.
- **Easier Debugging of Invalid Actions**—During testing, we observed that when the LLM was allowed to play multiple cards at once, it became difficult to identify the cause of any invalid actions. However, by playing one card at a time, if an invalid action occurs, we can analyze the LLM's weaknesses more precisely.

This paper does not specifically focus on analyzing invalid actions made by the LLM in MTG because we found this to be a vast and complex topic that extends beyond our current research scope. For example, we noticed that the most common invalid action was the LLM mistakenly believing it had enough mana to play a card. However, upon reviewing the LLM's CoT responses, we found that the LLM actually had a good understanding of its available mana, which was quite surprising. Moreover, the precise impact of invalid actions on LLMs remains unclear in our current understanding. Therefore, in future work, we will conduct a more detailed analysis of these invalid actions.

Finally, the difficulty arc demonstrates that a well-designed DDA prompt can enable players to achieve a comeback victory in the final stages. Therefore, we recommend using the difficulty arc as a metric to evaluate the effectiveness of DDAs and game mechanics in MTG.

8. Limitations and future work

Based on our observations of the LLM's CoT responses, we found that the LLM tends to plan its future moves according to its objectives and makes decisions based on that plan. A key direction for our future research is to investigate how strongly the LLM's reasoning correlates with its in-game decisions. To illustrate this with a simple example: if the LLM states, “*I want to summon a creature card that requires Red mana*”, and then decides to play a land card that generates Red mana, this would indicate a high level of correlation between its reasoning and actions in MTG. Conducting such in-depth analysis can provide better insights into the LLM's decision-making process and help refine its optimization. Additionally, in this study, we did not focus on investigating the impact of the invalid action threshold on game balance, as this constitutes a distinct and substantial research topic beyond the scope of this paper. This aspect could also be considered a potential direction for future research.

Secondly, this paper primarily focuses on investigating the potential applications of LLM-DDA in simplified complex card games by proposing a framework that enables LLMs to serve as dynamic difficulty adjusters through balancing life point disparities between human players and LLM opponents. We acknowledge that best practices in DDA analysis recommend validating the approach using a range of human player profiles — from novices to experts — to capture the variability in game-specific experience. In the present work, however, this aspect was not explicitly addressed, as all players are modeled using a uniform LLM configuration. While this choice was made to isolate and evaluate the impact of LLM-DDA under controlled conditions, it also represents a limitation. Future work will extend this framework by incorporating heterogeneous player profiles and refining the full MTG implementation to more comprehensively assess the system's adaptability and robustness.

Finally, our current approach of using two LLMs to simulate an MTG match requires a significant amount of time. Completing a full game takes approximately 30 to 40 min. Optimizing this process will be a key focus of our future work—specifically, improving the LLM's response speed while maintaining its current level of accuracy.

CRedit authorship contribution statement

Xiaoxu Li: Writing – original draft, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Zifan Ye:** Software, Investigation, Data curation, Conceptualization. **Yi Xia:** Validation, Methodology, Investigation. **Ruck Thawonmas:** Supervision, Conceptualization.

Declaration of competing interest

The authors have declared no conflict of interest

Data availability

Data will be made available on request.

References

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, *Adv. Neural Inf. Process. Syst.* 33 (2020) 1877–1901.
- [2] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al., Emergent abilities of large language models, 2022, arXiv preprint [arXiv:2206.07682](https://arxiv.org/abs/2206.07682).
- [3] Y. Xian, C.H. Lampert, B. Schiele, Z. Akata, Zero-shot learning—a comprehensive evaluation of the good, the bad and the ugly, *IEEE Trans. Pattern Anal. Mach. Intell.* 41 (9) (2018) 2251–2265.
- [4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q.V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, *Adv. Neural Inf. Process. Syst.* 35 (2022) 24824–24837.
- [5] M. Verma, S. Bhambri, S. Kambhampati, Preference proxies: Evaluating large language models in capturing human preferences in human-AI tasks, in: *ICML 2023 Workshop the Many Facets of Preference-Based Learning*, 2023.
- [6] S. Jorgensen, G. Nadizar, G. Pietropoli, L. Manzoni, E. Medvet, U.-M. O'Reilly, E. Hemberg, Large language model-based test case generation for GP agents, in: *Proceedings of the Genetic and Evolutionary Computation Conference*, 2024, pp. 914–923.
- [7] J.S. Park, J. O'Brien, C.J. Cai, M.R. Morris, P. Liang, M.S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–22.
- [8] K. Nottingham, Y. Razeghi, K. Kim, J. Lanier, P. Baldi, R. Fox, S. Singh, Selective perception: Learning concise state descriptions for language model actors, in: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, 2024, pp. 327–341.
- [9] Q.R. Yong, A. Mitchell, From playing the story to gaming the system: Repeat experiences of a large language model-based interactive story, in: L. Holloway-Attaway, J.T. Murray (Eds.), *Interactive Storytelling*, Springer Nature Switzerland, Cham, 2023, pp. 395–409.
- [10] T. Ashby, B.K. Webb, G. Knapp, J. Searle, N. Fulda, Personalized quest and dialogue generation in role-playing games: A knowledge graph-and language model-based approach, in: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 2023, pp. 1–20.
- [11] N. Akoury, Q. Yang, M. Iyyer, A framework for exploring player perceptions of llm-generated dialogue in commercial video games, in: *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 2295–2311.
- [12] P. Sweetser, Large language models and video games: A preliminary scoping review, in: *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, 2024, pp. 1–8.
- [13] X. Xu, Y. Wang, C. Xu, Z. Ding, J. Jiang, Z. Ding, B.F. Karlsson, A survey on game playing agents and large models: Methods, applications, and challenges, 2024, arXiv preprint [arXiv:2403.10249](https://arxiv.org/abs/2403.10249).
- [14] S. Zhai, H. Bai, Z. Lin, J. Pan, P. Tong, Y. Zhou, A. Suhr, S. Xie, Y. LeCun, Y. Ma, et al., Fine-tuning large vision-language models as decision-making agents via reinforcement learning, *Adv. Neural Inf. Process. Syst.* 37 (2024) 110935–110971.
- [15] P. Chen, P. Bu, J. Song, Y. Gao, B. Zheng, Can VLMs play action role-playing games? Take black Myth Wukong as a study case, in: *NeurIPS 2024 Workshop on Open-World Agents*, 2024.
- [16] X. Li, Y. Xia, R. Thawonmas, Personalized game difficulty with large language models: a preliminary study, in: *2025 IEEE International Conference on Consumer Electronics, ICCE, IEEE*, 2025, pp. 1–6.
- [17] O. Topsakal, J.B. Harper, Benchmarking large language model (LLM) performance for game playing via Tic-Tac-Toe, *Electronics* 13 (8) (2024) 1532.
- [18] A. Zhu, L. Martin, A. Head, C. Callison-Burch, CALYPSO: LLMs as Dungeon Master's assistants, in: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 19, 2023, pp. 380–390.
- [19] M. Brienza, E. Musumeci, V. Suriani, D. Affinita, A. Pennisi, D. Nardi, D.D. Bloisi, LLMCoach: Generating robot soccer plans using multi-role large language models, 2024, arXiv preprint [arXiv:2406.18285](https://arxiv.org/abs/2406.18285).
- [20] Z. Qin, H. Wang, D. Liu, Z. Song, C. Fan, Z. Lv, J. Wu, Z. Lei, Z. Tu, D. Chu, et al., UNO arena for evaluating sequential decision-making capability of large language models, in: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 7630–7645.
- [21] Y. Yim, C. Chan, T. Shi, Z. Deng, W. Fan, T. Zheng, Y. Song, Evaluating and enhancing llms agent based on theory of mind in guandan: A multi-player cooperative game under imperfect information, 2024, arXiv preprint [arXiv:2408.02559](https://arxiv.org/abs/2408.02559).
- [22] C. Huang, Y. Cao, Y. Wen, T. Zhou, Y. Zhang, PokerGPT: An end-to-end lightweight solver for multi-player texas hold'em via large language model, 2024, arXiv preprint [arXiv:2401.06781](https://arxiv.org/abs/2401.06781).
- [23] S. Hu, T. Huang, L. Liu, Pokéllmon: A human-parity agent for pokémon battles with large language models, 2024, arXiv preprint [arXiv:2402.01118](https://arxiv.org/abs/2402.01118).
- [24] P.I. Cowling, C.D. Ward, E.J. Powley, Ensemble determinization in monte carlo tree search for the imperfect information card game magic: The gathering, *IEEE Trans. Comput. Intell. AI Games* 4 (4) (2012) 241–257.
- [25] H.N. Ward, B. Mills, D.J. Brooks, D. Troha, A.S. Khakhalin, AI solutions for drafting in magic: the gathering, in: *2021 IEEE Conference on Games (CoG)*, 2021, pp. 1–8, <http://dx.doi.org/10.1109/CoG52621.2021.9619100>.
- [26] F. Zhu, The intelligence of player habits and reflexivity in magic: The gathering arena limited draft, *Angelaki* 28 (3) (2023) 38–55.

- [27] T. Bertram, J. Fürnkranz, M. Müller, Predicting human card selection in magic: The gathering with contextual preference ranking, in: 2021 IEEE Conference on Games (CoG), IEEE, 2021, pp. 1–8.
- [28] M.D. Sebag, “A definition of enchantment”: A new approach to the analysis of magic systems in fantasy games, *Games Cult.* (2025) 15554120241310925.
- [29] K. Chatterjee, R. Ibsen-Jensen, The complexity of deciding legality of a single step of magic: The gathering, in: *ECAI 2016*, IOS Press, 2016, pp. 1432–1439.
- [30] J. Tromp, G. Farneback, Combinatorics of go, in: *International Conference on Computers and Games*, Springer, 2006, pp. 84–99.
- [31] R. Thawonmas, T. Harada, Ai for game spectators: Rise of ppg, in: *Workshops At the Thirty-First AAAI Conference on Artificial Intelligence*, 2017.
- [32] G. Costikyan, *Uncertainty in Games*, vol. 2, The MIT Press, 2013.
- [33] M. Csikszentmihalyi, *Flow: The psychology of optimal experience*, Haper Row (1990).